

ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ · ЛЦТ 2026

CRM ИТ Школы Ростелекома

Система контроля взаимодействия с вузами по ИТ-направлениям: 14 этапов со сроками, аналитика, отчёты, ИИ-помощник и Telegram-уведомления.

React 19

Vite

FastAPI

PostgreSQL

Keycloak

Ollama

Telegram

Docker

14

этапов работы с вузом
в 4 фазах

3

роли: менеджер,
руководитель,
администратор

150

пользователей под
нагрузкой — 0 ошибок

AES-256

шифрование
персональных данных,
152-ФЗ

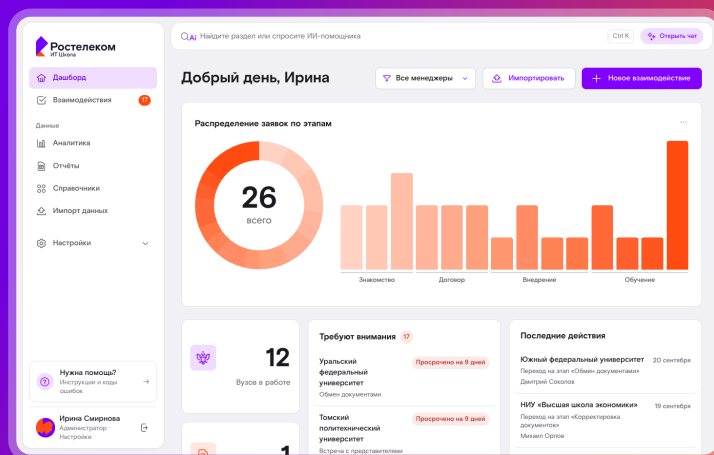
Попробовать

Сервис rtk-itschool.ru

Логин [testuser](#)

Пароль [ItSchool-2026!](#)

Код [GitHub](#)



Содержание

Продукт

Процесс

Курс новичка

ИИ-помощник

Поиск

Доска этапов

Карточка вуза

Аналитика

Экспорт графиков

Отчёты

Импорт из Excel

Администрирование

Профиль и Telegram

Справка и ошибки

Мобильная версия

Система

Требования ТЗ → где реализовано

Стек

Архитектура

Данные и синхронизация

Безопасность и роли

Защита персональных данных (152-ФЗ и ФСТЭК № 117)

ИИ-помощник изнутри

Telegram-бот изнутри

База данных

API

Эксплуатация

Развёртывание на сервере

Keycloak

Telegram-бот

Переменные окружения

Локальный запуск

Тесты

Библиотеки и компоненты

Структура репозитория

Продукт

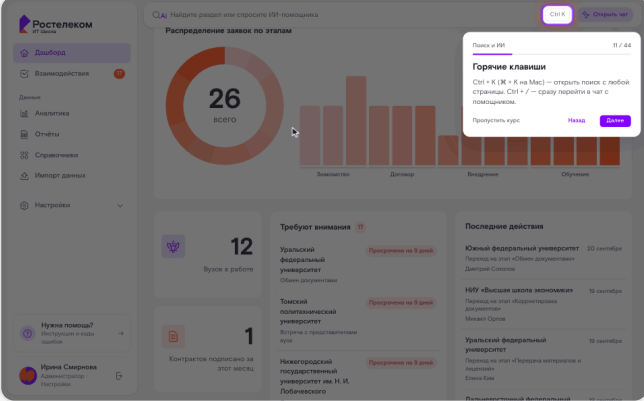
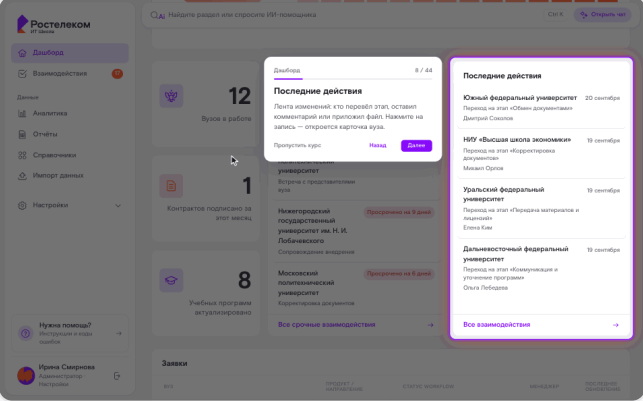
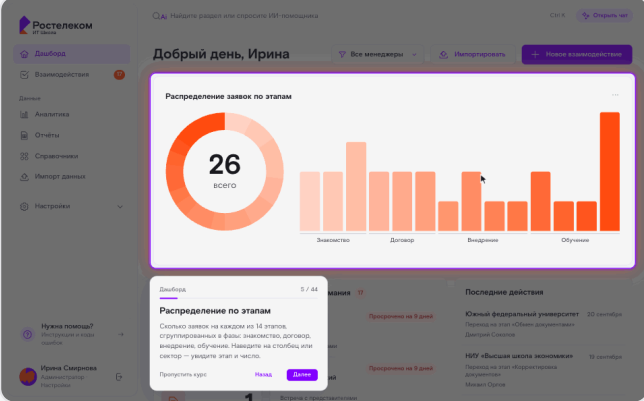
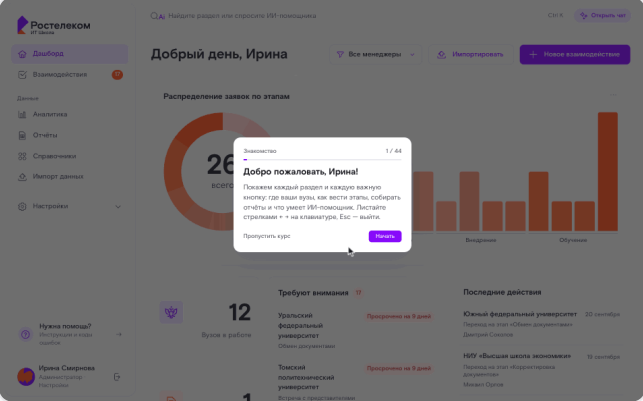
Процесс

Взаимодействие = вуз + ИТ-направление + программа + продукт. Оно проходит 14 этапов в 4 фазах, у каждого этапа есть срок и подсказка. Вперёд — свободно; назад и пропуск необязательного этапа — только с комментарием (`logic/Этапы.md`).

Знакомство	Договор	Внедрение	Обучение
1–3: контакты, коммуникация, встреча	4–6: документы, <i>корректировка</i> , подписание	7–10: материалы, сопровождение, обучение преподавателей, программа	11–14: занятия, документация, квалификация, контроль

Курс новичка

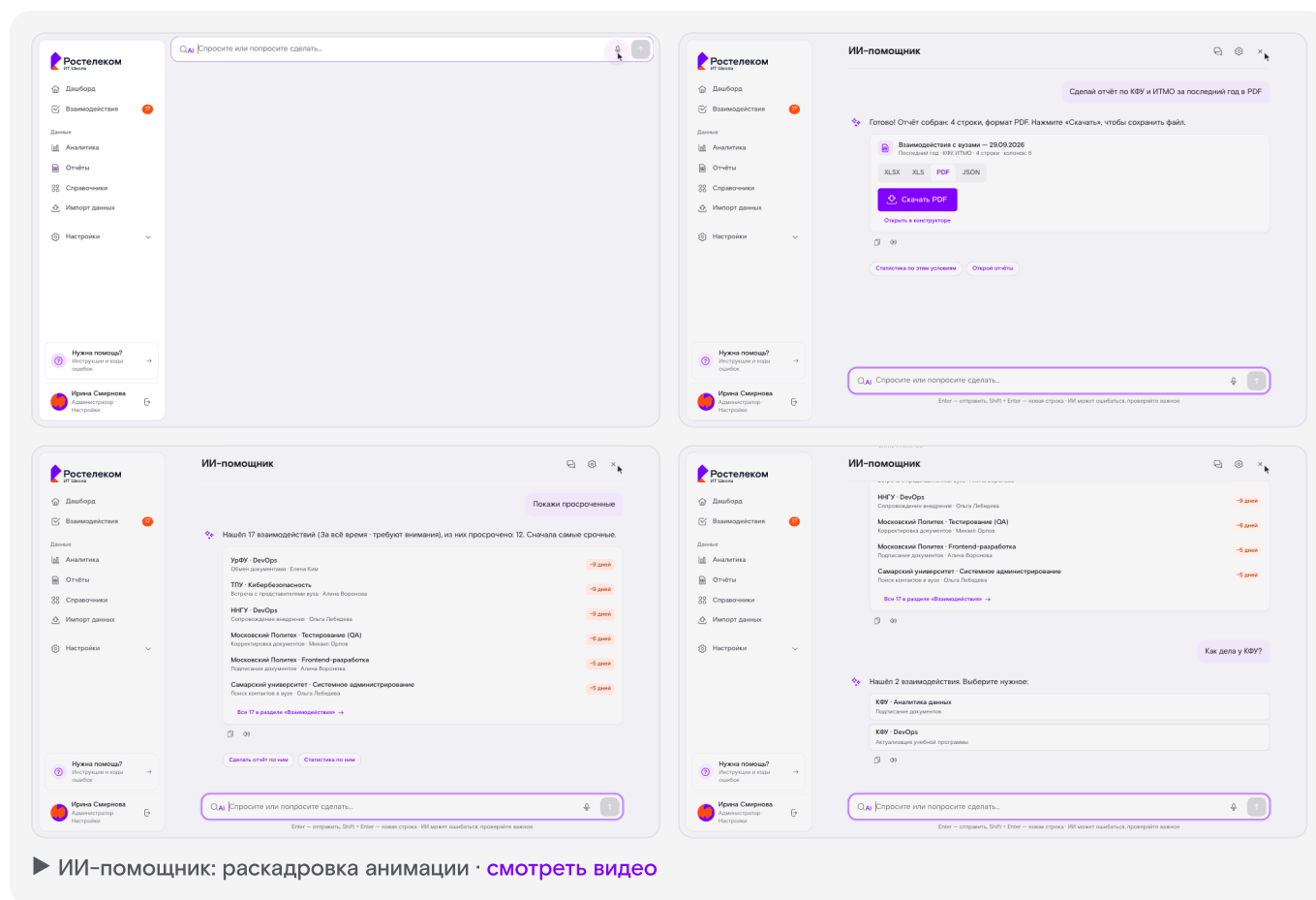
При первом входе открывается интерактивный тур: 44 шага по 10 разделам. Тур сам переходит по страницам, подсвечивает элемент и показывает анимацию: курсор жмёт кнопку, «печатается» запрос. Шаги зависят от роли, ← → листают, Esc — выход, прогресс хранится на сервере.



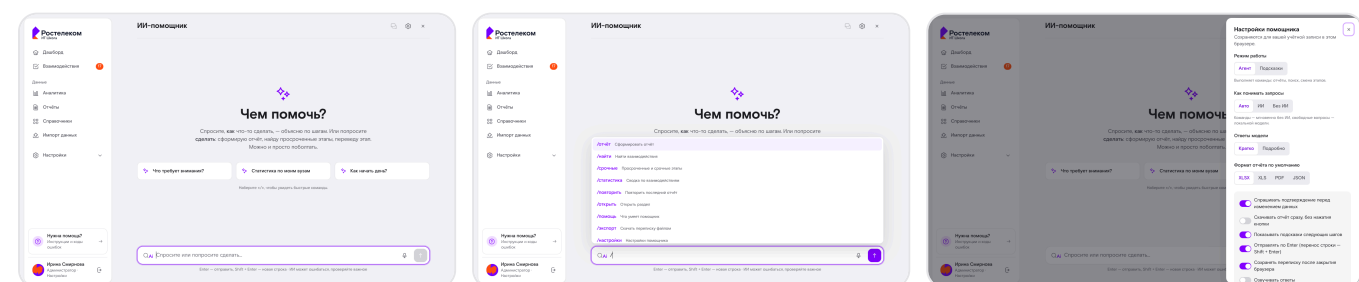
► Курс новичка: раскадровка анимации · [смотреть видео](#)

ИИ-помощник

Строка поиска превращается в чат. Помощник объясняет, как что-то сделать, или **делает сам**: отчёты, списки, сводки, смену этапа (с подтверждением и отменой). Понимает вузы в любом падеже, периоды словами, «мои», «кроме», «по ним». Есть быстрые команды **/**, голосовой ввод и озвучка.

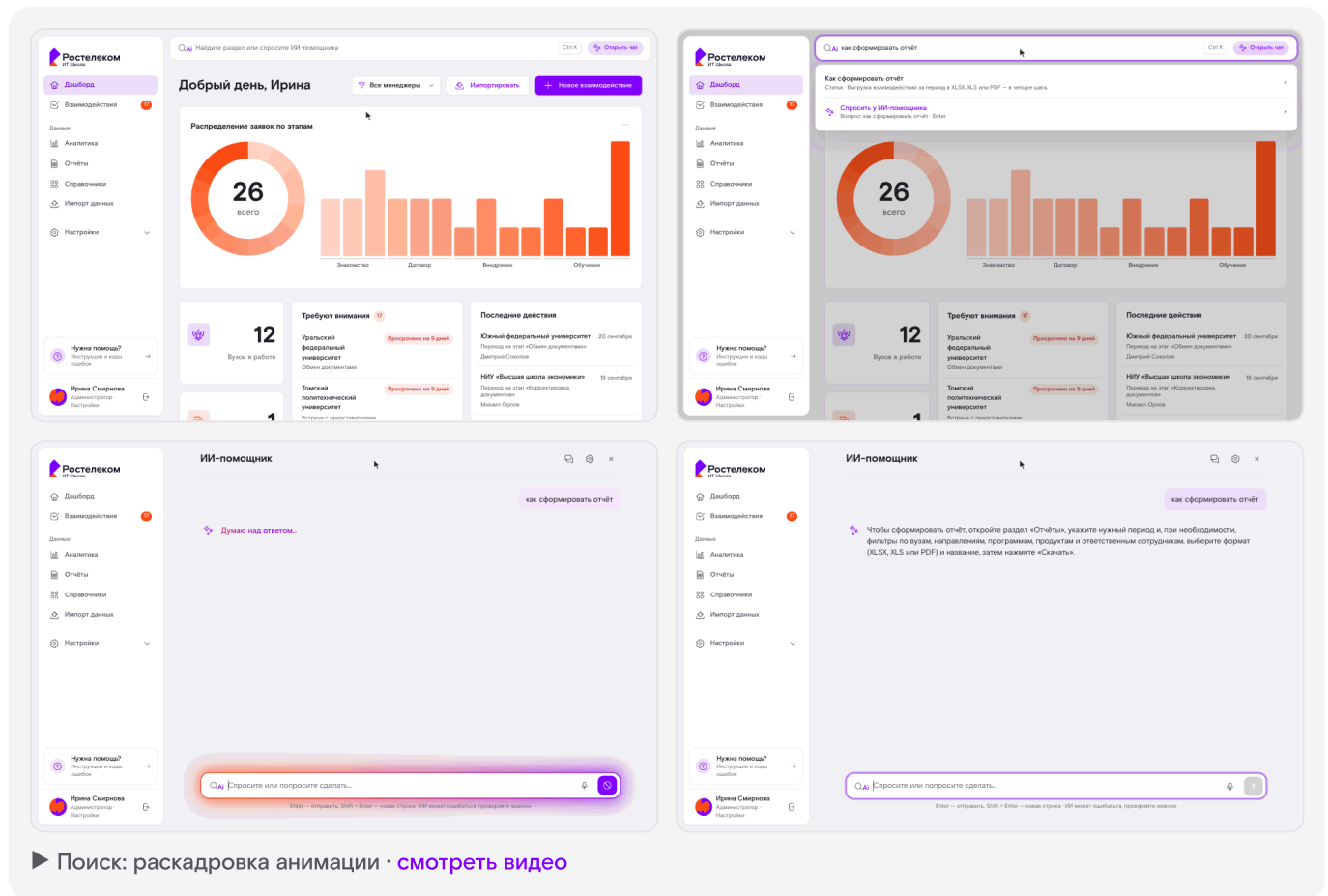


Запрос	Результат
«Отчёт по КФУ и ИТМО за год в PDF»	Карточка с форматом и кнопкой «Скачать»
«Покажи просроченные»	Список со сроками, строка открывает карточку
«Как дела у КФУ?», «Контакты ИТМО»	Этап, срок, ответственный, что делать; контакты
«Переведи КФУ на следующий этап»	«Было → станет», подтверждение, «Отменить»
«Статистика за год», «Нагрузка по менеджерам»	Сводка по фазам и людям



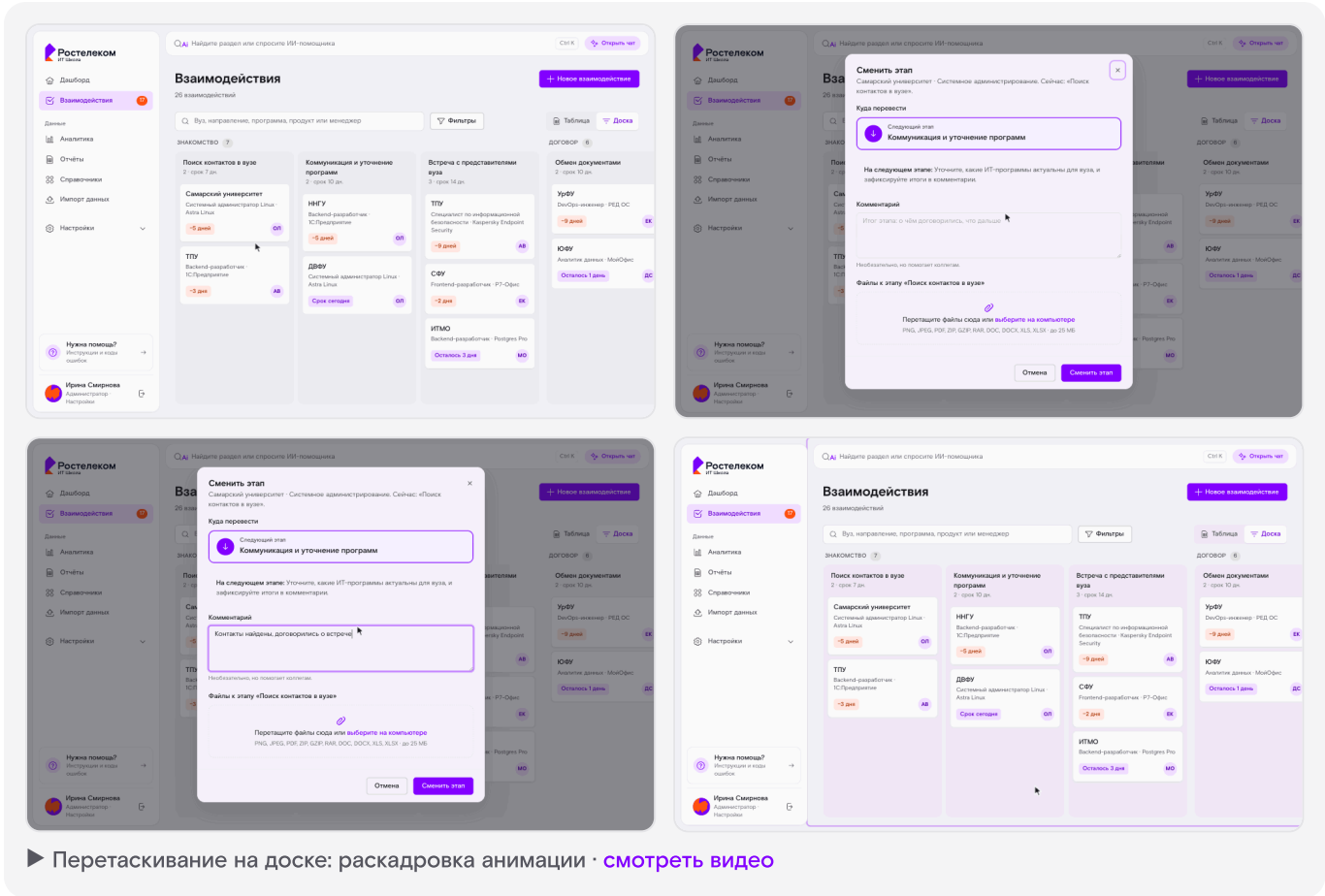
Поиск

Ctrl + K ищет разделы, блоки страниц и статьи справки. Enter отправляет вопрос помощнику, Ctrl + / открывает чат.

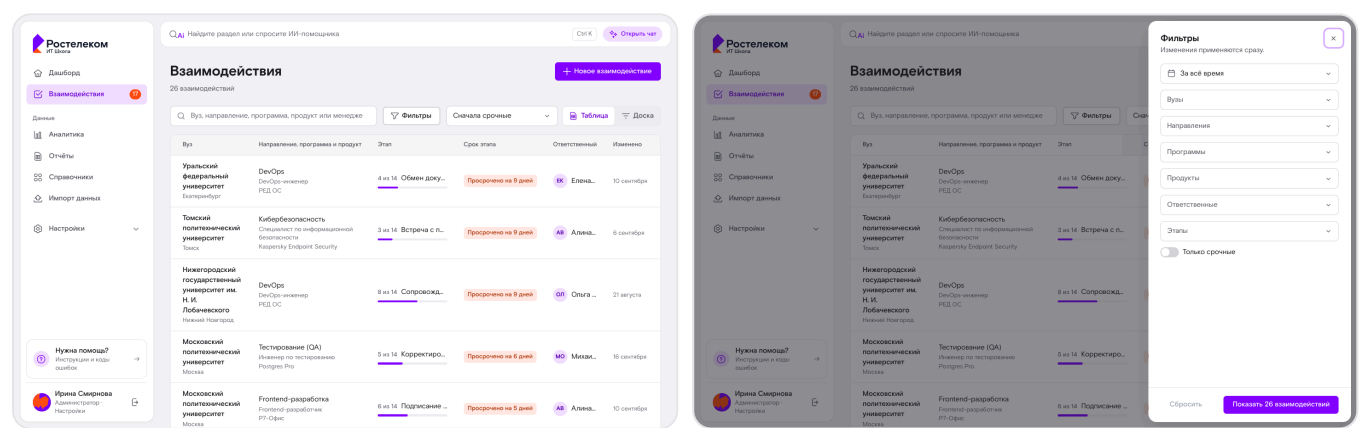


Доска этапов

Канбан по фазам: карточку перетаскивают на следующий этап, недопустимые колонки гаснут. Есть табличный вид с сортировкой. Фильтры по периоду, вузам, направлениям, продуктам, ответственным и срочности запоминаются.

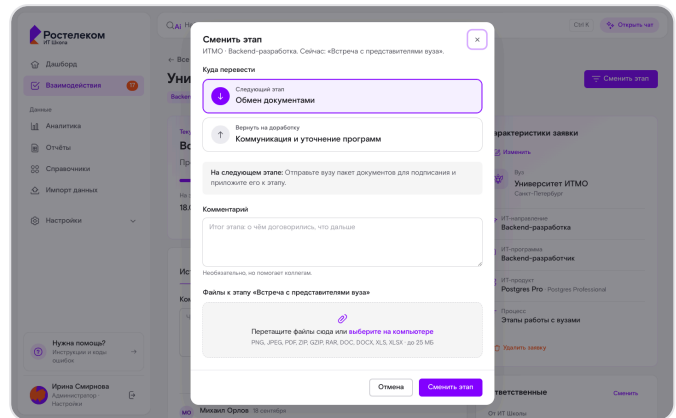
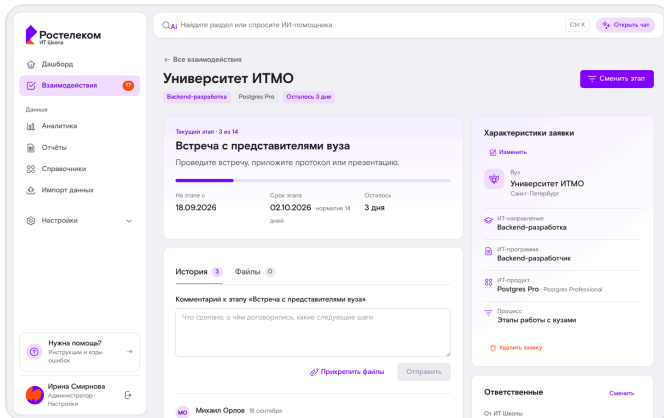


► Перетаскивание на доске: раскадровка анимации · [смотреть видео](#)



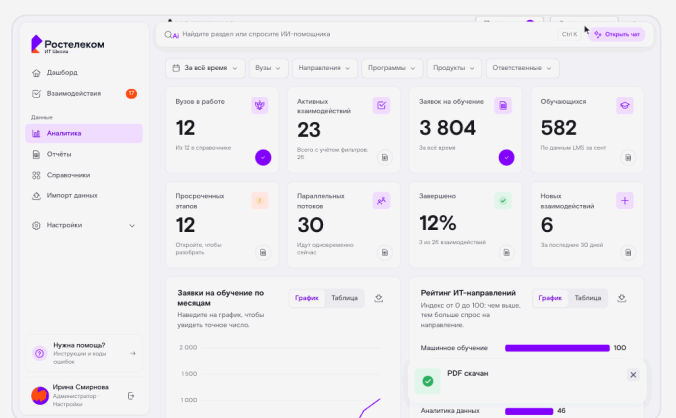
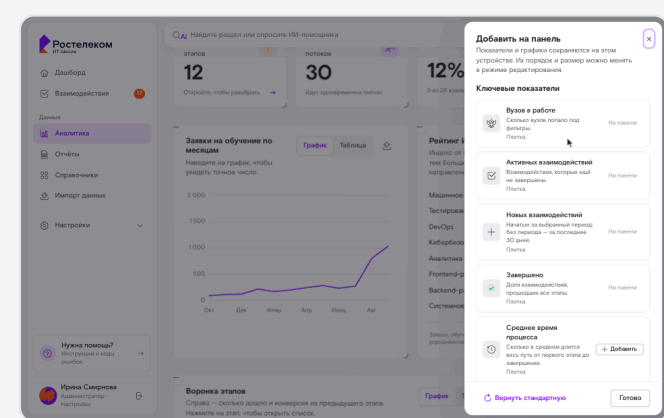
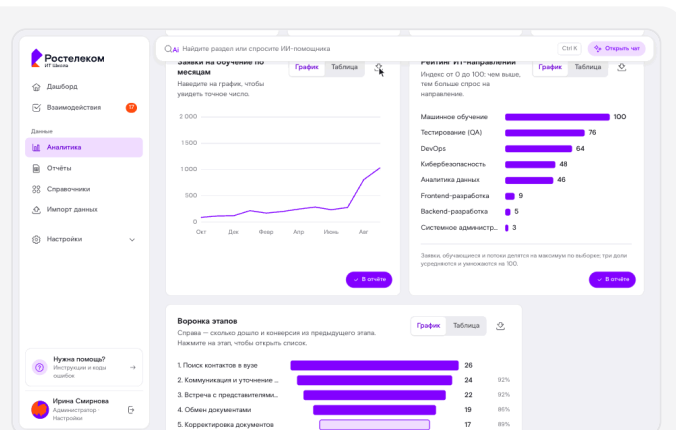
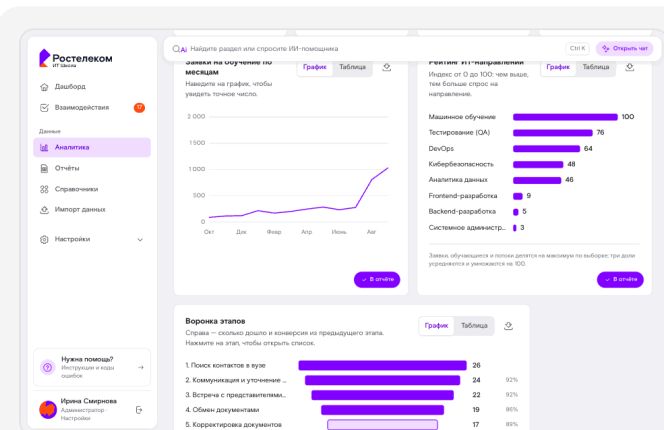
Карточка вуза

Текущий этап с прогрессом и остатком дней, характеристики, ответственные, путь по этапам, история, файлы по этапам. Смену этапа, комментарий и правки можно отменить из уведомления.

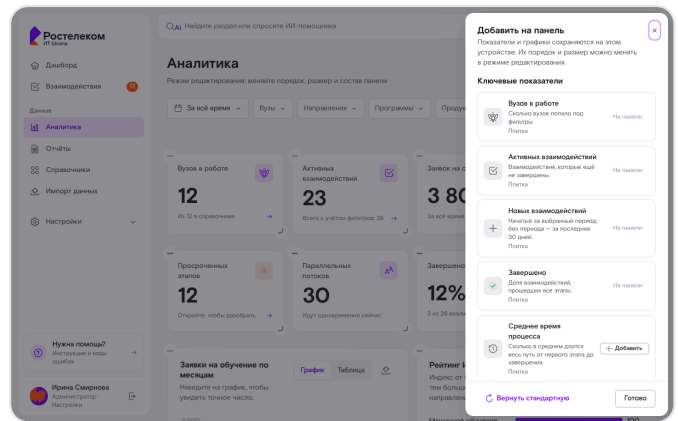
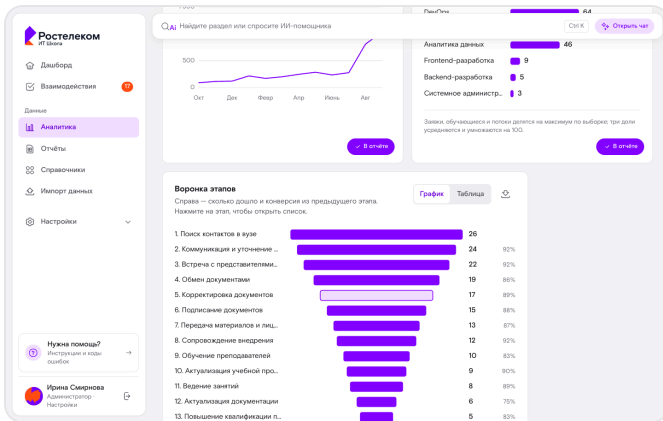


Аналитика

Панель-конструктор: виджеты перетаскиваются, меняют размер, добавляются из каталога. У каждого графика есть переключатель «График ⇌ Таблица». Все 11 типов графиков — собственные SVG без библиотек. Кнопка «В отчёт» добавляет карточку в PDF-отчёт.



► Аналитика: раскадровка анимации · [смотреть видео](#)

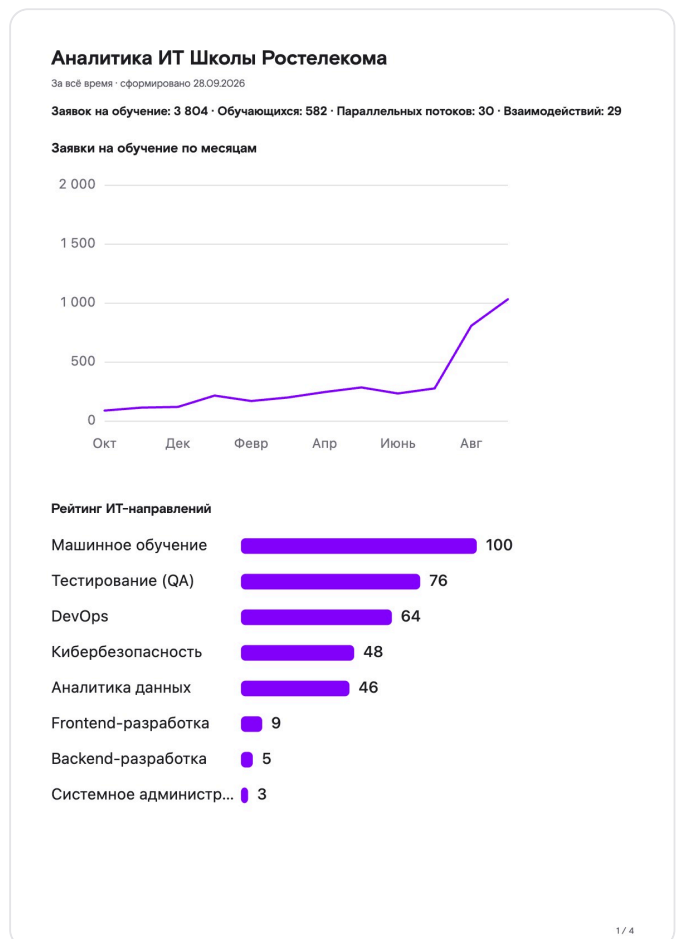


Экспорт графиков

Любой экспорт — настоящий файл без диалога печати. PNG графика рисуется в двойном разрешении с фирменными цветами и шрифтом, PDF всей панели — многостраничный. PDF, XLSX и ZIP генерируются кодом проекта ([lib/export](#)). Ниже — сами скачанные файлы:



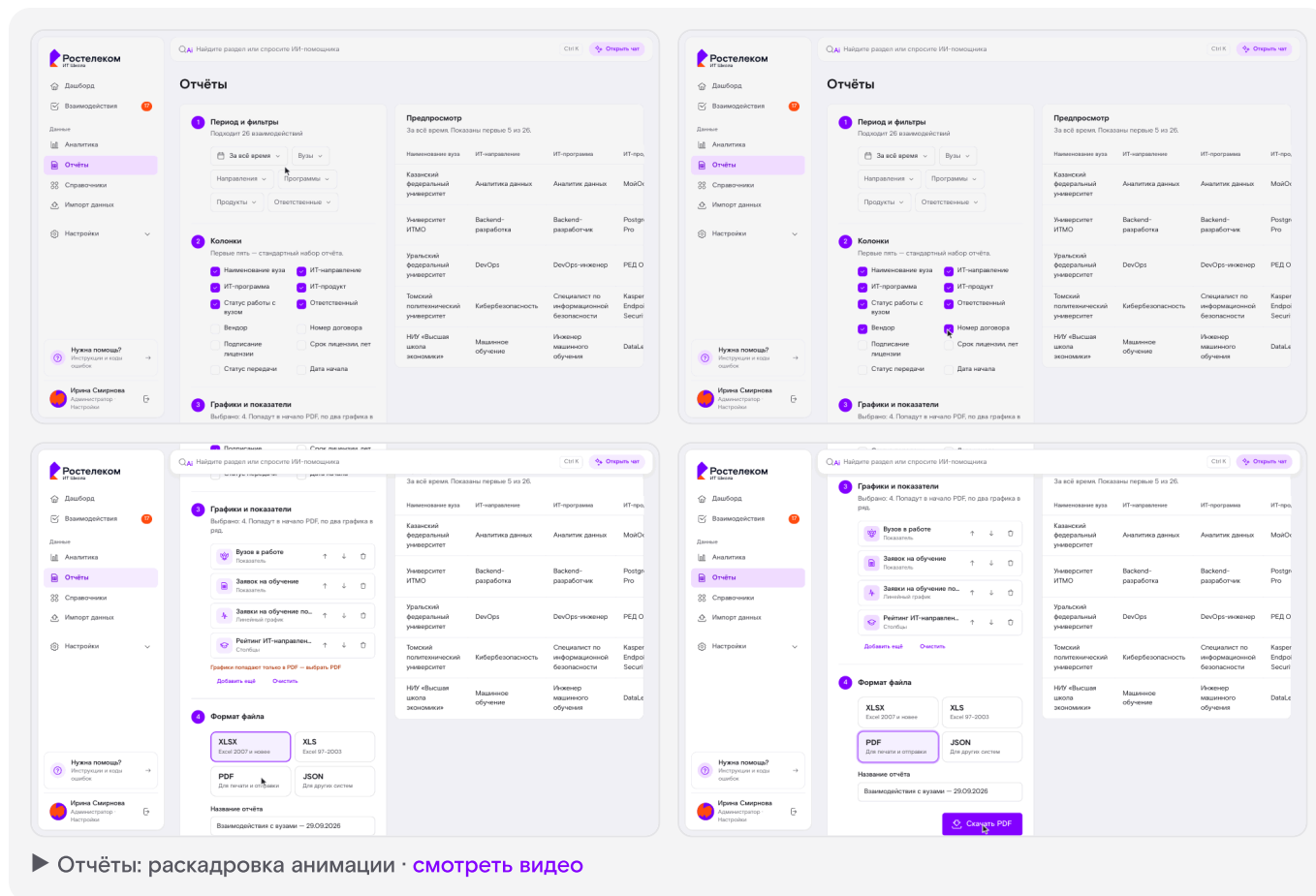
PNG графика



PDF панели, стр. 1 из 4

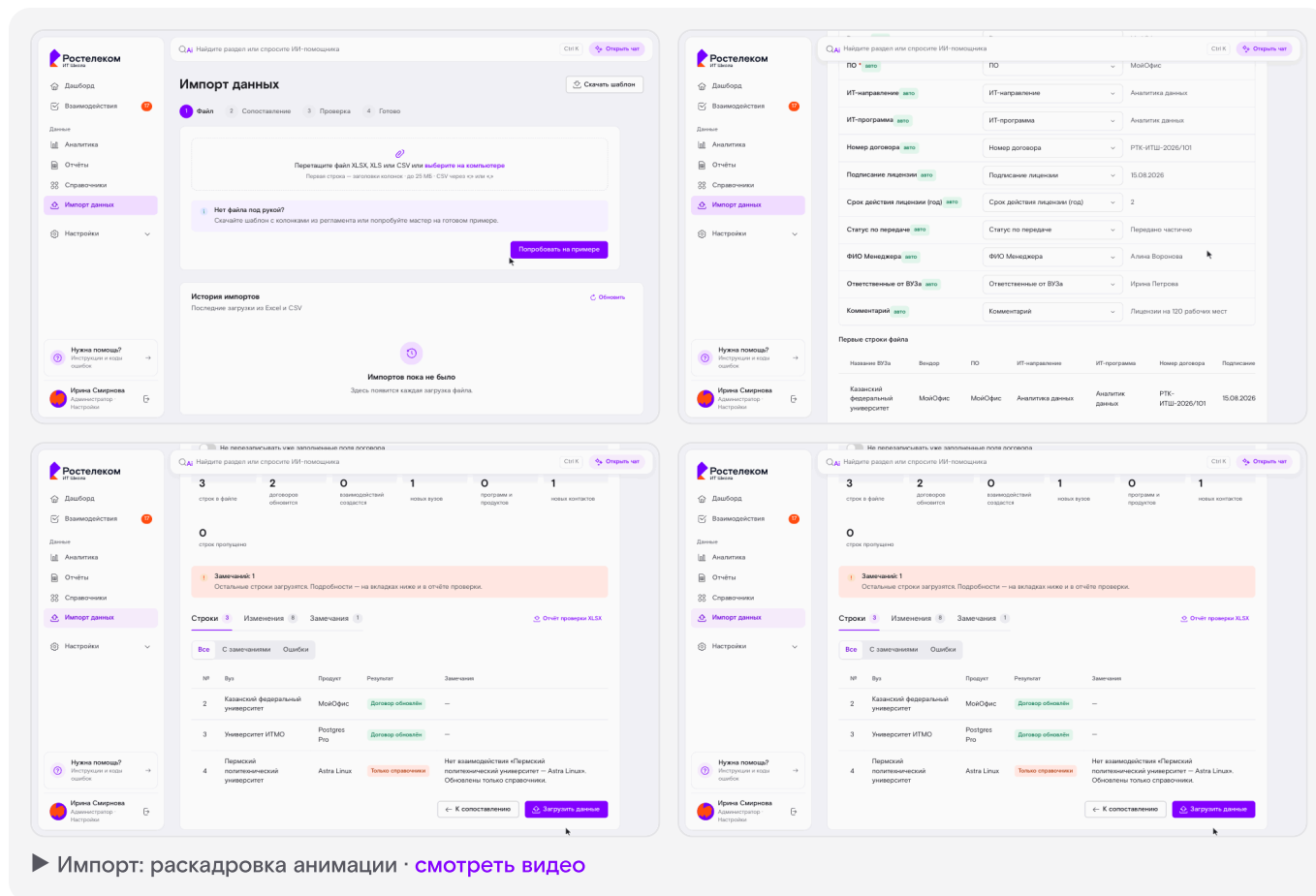
Отчёты

4 шага: фильтры → колонки → графики из «Аналитики» → формат (XLSX / XLS / PDF / JSON). Справа — живой предпросмотр. JSON — результирующий файл для других систем: описание колонок и строки-объекты. История позволяет пересобрать любой отчёт по свежим данным.

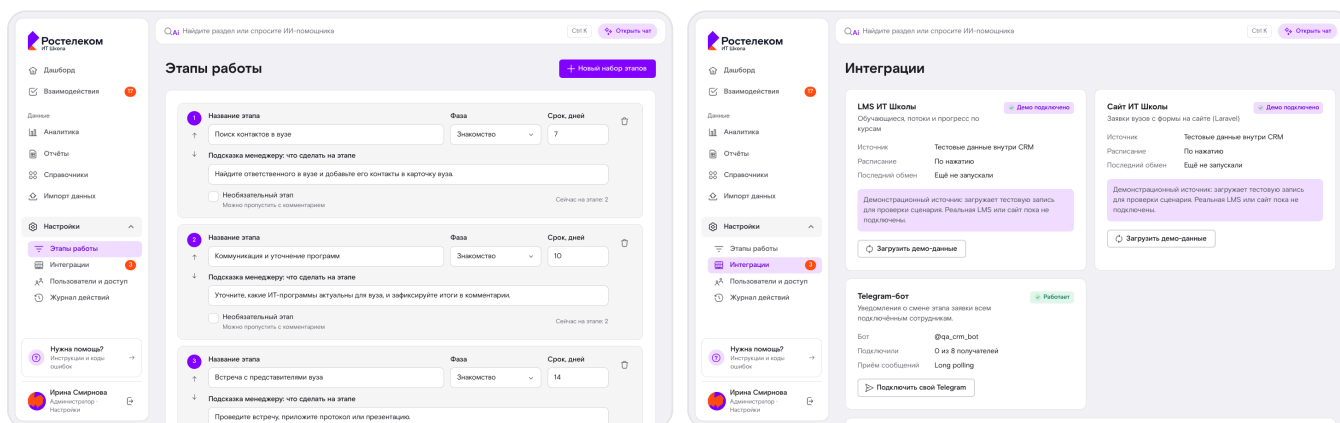


Импорт из Excel

Мастер: файл (xlsx / xls / csv) → сопоставление колонок → проверка → загрузка. Есть шаблон, пример и история; импорт откатывается целиком.

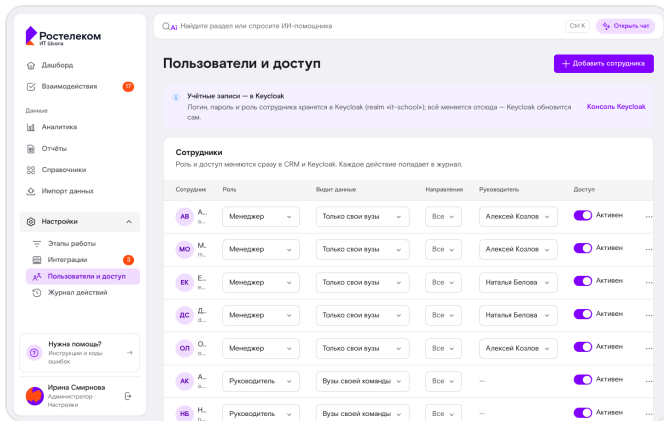


Администрирование

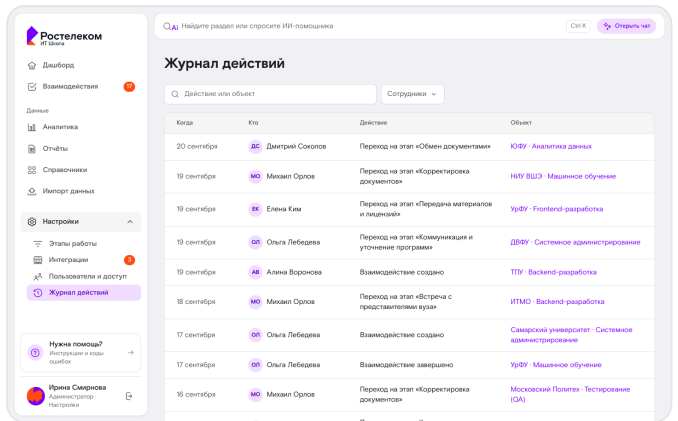


Этапы — названия, порядок, сроки, необязательные этапы

Интеграции — LMS и сайт, разбор входящих записей



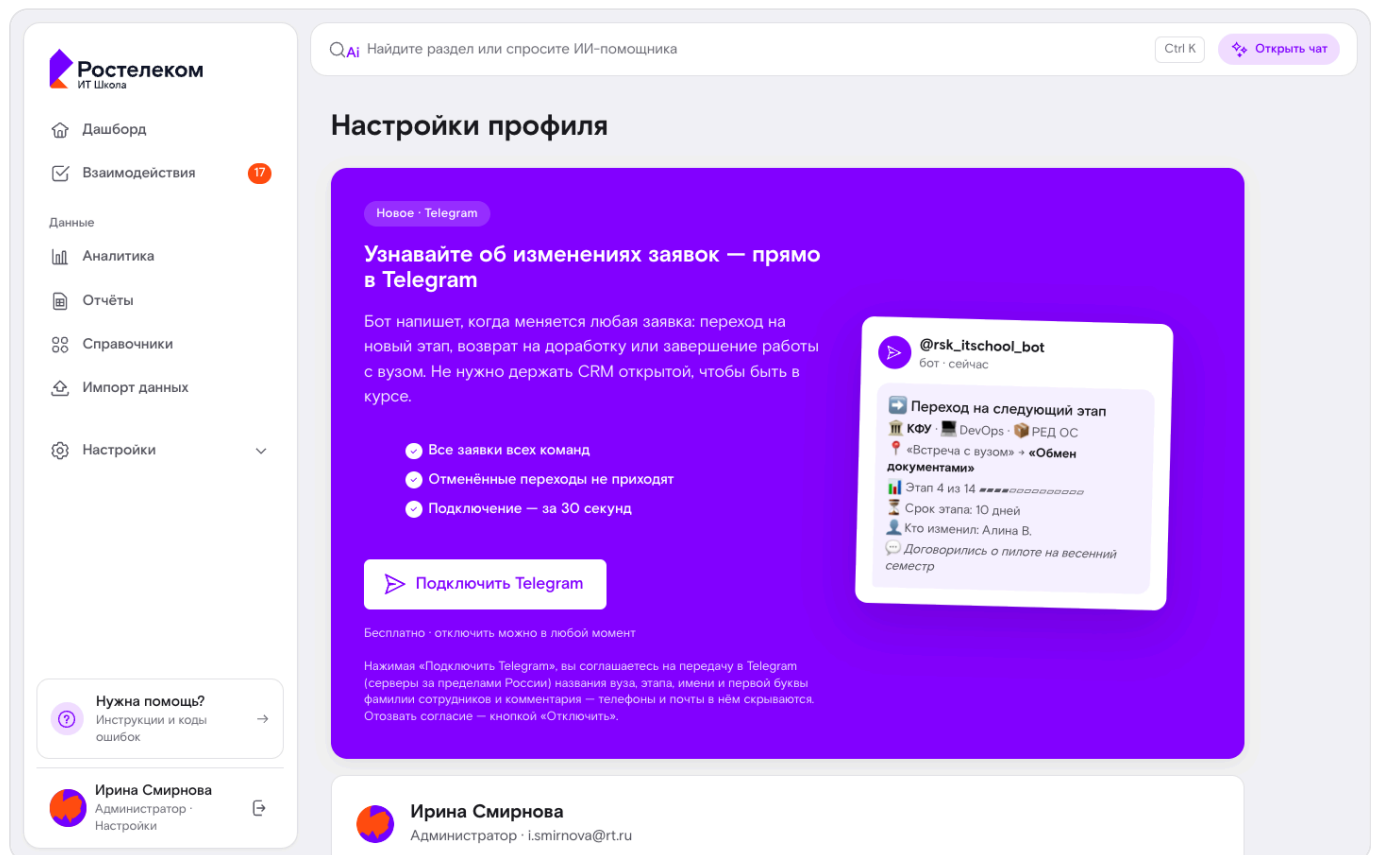
Пользователи — роли, видимость, блокировка



Журнал — кто, что и когда менял

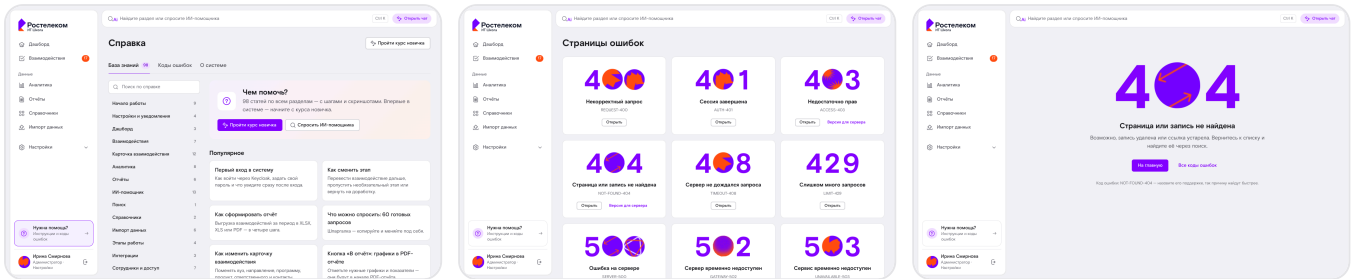
Профиль и Telegram

Настройки хранятся на сервере: аватар, масштаб, анимации, стартовая страница, горячие клавиши. Бот [@rsk_itschool_bot](#) подключается за 30 секунд и присылает переходы этапов.



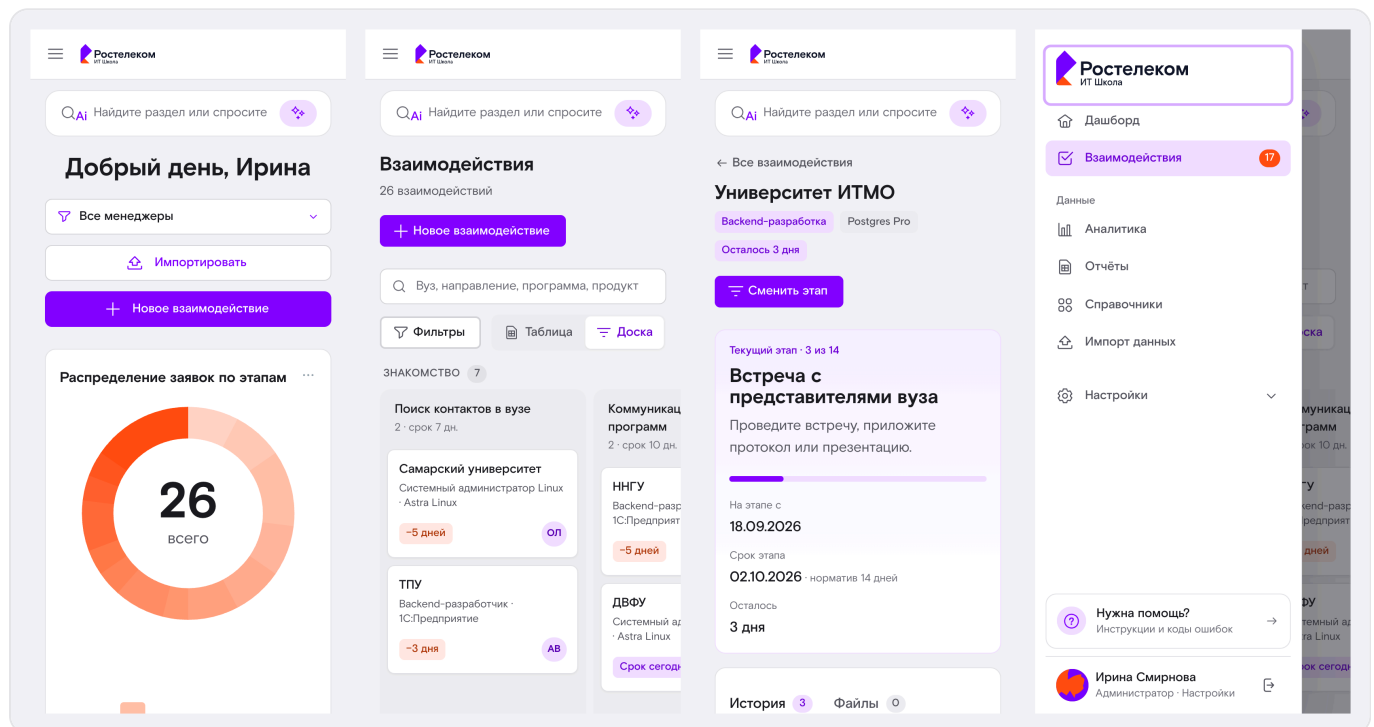
Справка и ошибки

Статьи со скриншотами, 60 готовых запросов для помощника, кнопка «Пройти курс заново». У каждой ошибки есть код и описание; страницы 404 / 503 оформлены в фирменном стиле.



Мобильная версия

Все экраны работают от 360 рх: таблицы превращаются в карточки, диалоги — в нижние листы, меню открывается по ☰.



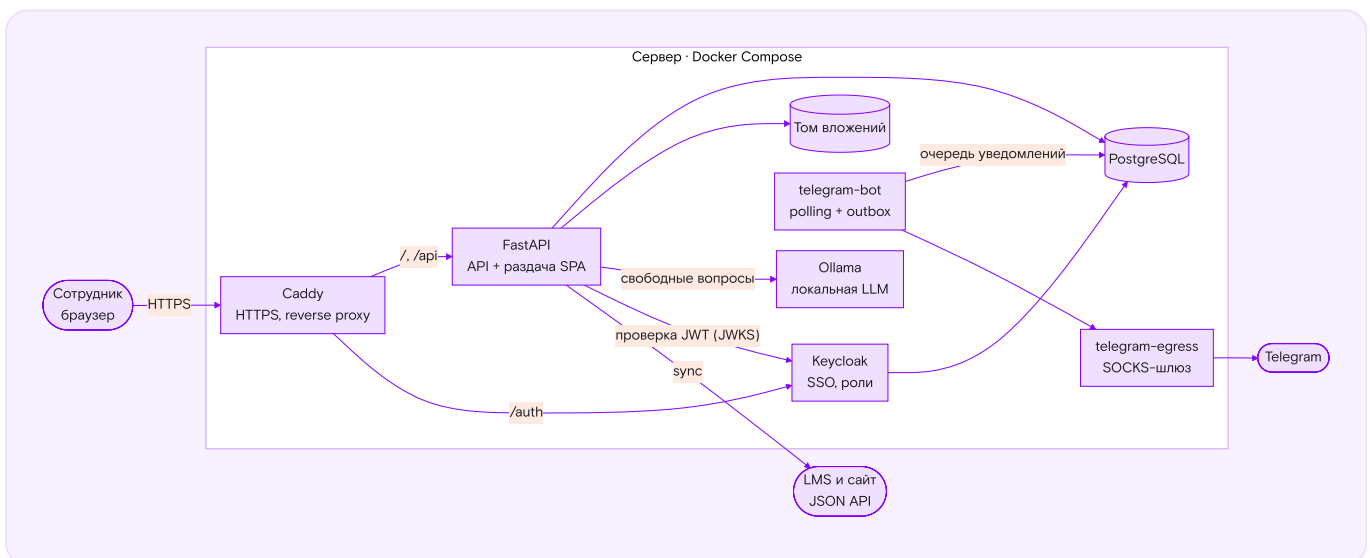
Требования ТЗ → где реализовано

Требование ТЗ	Где в продукте	Где в коде
Каталоги вузов, направлений, продуктов, ответственных; загрузка xls/xlsx по маппингу полей	«Справочники», «Импорт данных»	features/catalogs , domain/import.js , lib/xlsx
Путь взаимодействия, переходы между статусами с комментарием, файлы в статусах	Карточка, доска этапов	features/interactions , services/attachment.py
Создание и изменение workflow, переименование статусов	«Этапы работы»	features/workflows
Фильтрация за период по вузам, направлениям, продуктам, ответственным	Панель фильтров	features/filters , domain/filters.js
Статистика, диаграммы и графики в PNG/PDF	«Аналитика»	features/analytics , ui/charts , lib/export
Отчёты xls/xlsx/pdf по выбранным колонкам; результирующий JSON	«Отчёты»	features/reports , lib/export
Данные LMS и сайта по API (JSON) → существующий или новый workflow	«Интеграции»	services/integration.py , features/integrations
Keuscloak, роли «пользователь / руководитель / администратор», видимость данных	Вход, «Пользователи и доступ»	core/security.py , domain/state.py , domain/roles.js
Кэш действий, работа без перезагрузки страницы, отклик ≤ 1 с	Везде	store/persistence.js , store/rebase.js
Коды ошибок	«Справка → Коды ошибок», страницы 404 / 503	domain/errors.js , features/errors
Документация со скриншотами внутри платформы (пользователь и администратор)	«Справка»	features/help
152-ФЗ, приказ ФСТЭК № 117	См. Защита ПДн	core/crypto.py , core/security_headers.py
50 пользователей, 10 параллельных отчётов	Проверено: 150 пользователей и 15 отчётов, 0 ошибок	—
Swagger, перечень библиотек, Docker, Linux, модель в Archi	/docs , Библиотеки , deploy/ , docs/architecture	—

Стек

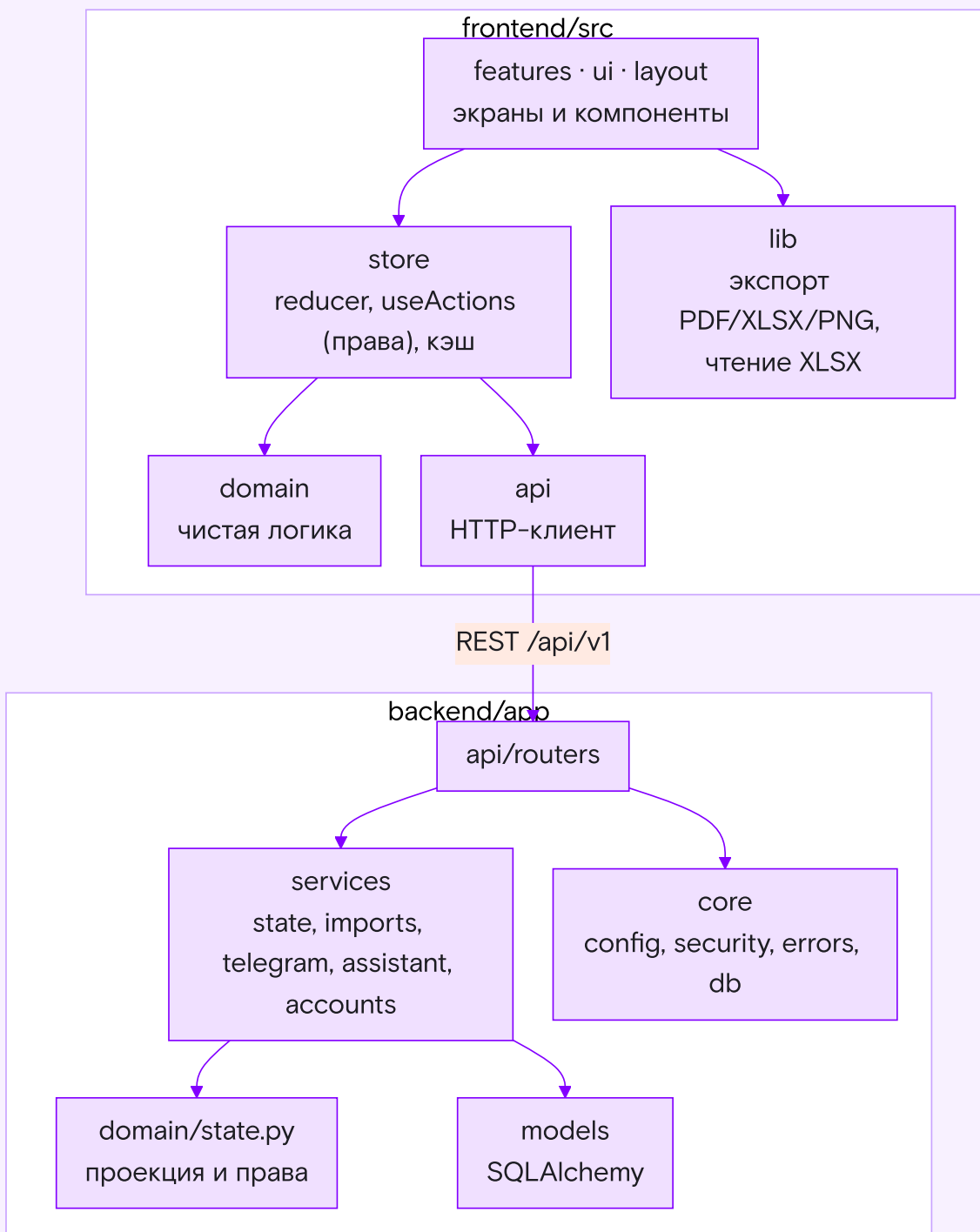
Слой	Технологии
Интерфейс	React 19, Vite 8, дизайн-система Ростелекома (Атомаро), шрифт Rostelecom Basis, GSAP; графики, PDF, XLSX, XLS — собственный код
API	Python 3.12, FastAPI, Pydantic, SQLAlchemy 2, Alembic
Данные	PostgreSQL 16 (локально — SQLite), шифрование AES-256-GCM
Вход и роли	Keycloak 26 (OIDC, Authorization Code + PKCE)
ИИ	Ollama с локальной моделью Qwen — данные не покидают сервер
Уведомления	Telegram Bot API: отдельный контейнер бота, очередь в PostgreSQL
Инфраструктура	Docker Compose, Caddy (HTTPS, Let's Encrypt), GitHub Actions, Linux (Ubuntu)

Архитектура



Модель в Archi (ArchiMate 3: функциональная и компонентная архитектура, требования 152-ФЗ / ФСТЭК № 117) — [docs/architecture/rtk-it-school.archimate](#), открывается в Archi через *File* → *Open*. Генерируется из [generate_archi.py](#).

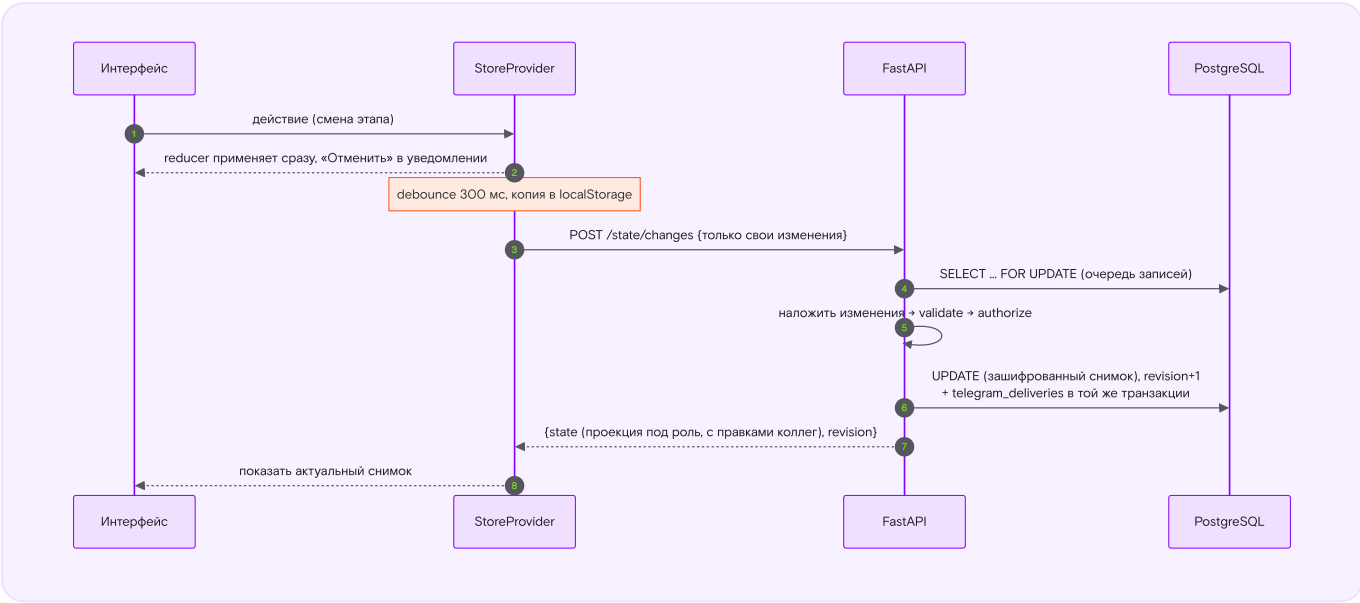
- **SPA-first.** Бизнес-логика (этапы, сроки, фильтры, отчёты, аналитика, разбор команд помощника) живёт в `frontend/src/domain` — чистом JS без React, покрытом тестами. Интерфейс отвечает мгновенно и работает без сети.
- **Сервер** отвечает за то, что должно быть надёжным: авторизацию, проверку прав на каждое изменение, фильтрацию данных под роль, версионирование, файлы, импорт с откатом, интеграции, LLM и доставку уведомлений.
- **Всё в одном origin:** FastAPI отдаёт и `/api/v1`, и собранный фронтенд, поэтому CORS в продакшене не нужен.



Данные и синхронизация

Состояние CRM — один версионированный JSON-снимок (`state_snapshots`), в базе он хранится зашифрованным. Интерфейс меняет свою копию сразу, а на сервер отправляет **только свои изменения** относительно последнего принятого снимка (`diffState` → `POST /state/changes`). Сервер под блокировкой строки накладывает их на актуальный снимок, проверяет права и целостность и

возвращает снимок со всеми изменениями коллег. Поэтому одновременная работа не конфликтует и не затирает чужие правки: нагрузочный тест на 150 пользователей — 0 потерянных изменений.



- **Офлайн:** при потере сети изменения остаются в кэше и отправляются после восстановления.
- **Проекция под роль:** `GET /state` отдаёт менеджеру только его взаимодействия, руководителю — только команду (`project_state_for_principal`).
- **Импорт** применяется одной транзакцией: снимок до импорта сохраняется в `import_jobs` для отката.

Безопасность и роли

Роль	Права
Менеджер	Свои взаимодействия: этапы, комментарии, файлы, отчёты
Руководитель	Команда: ответственные, этапы, импорт, интеграции, менеджеры команды, журнал
Администратор	Всё, включая роли и видимость данных

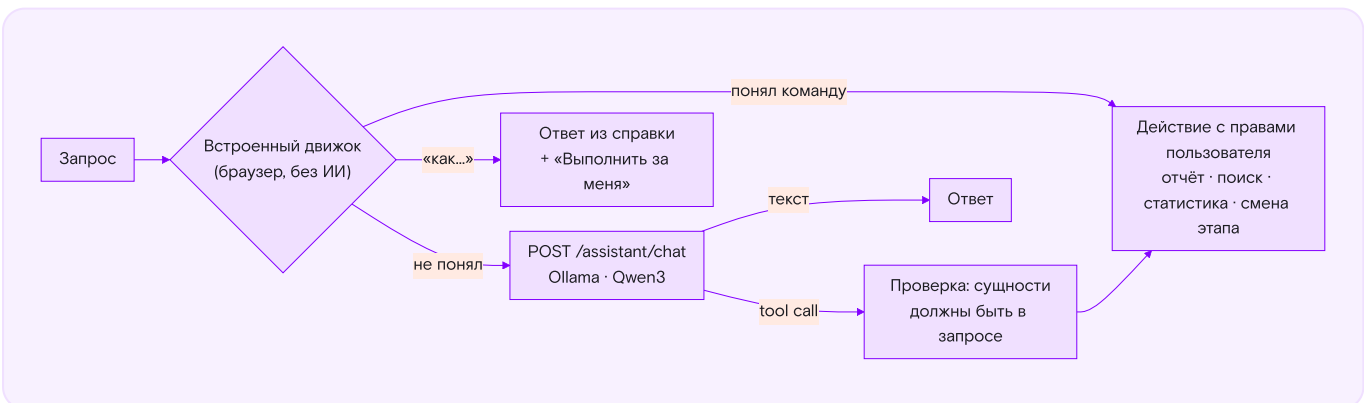
- **Вход:** Keycloak, Authorization Code + PKCE (без client secret в браузере). API проверяет RS256-токен по JWKS, роли Keycloak `KAM` / `MANAGER_LEAD` / `ADMIN` отображаются в роли приложения. Для стенда есть демо-вход (HS256 JWT): в `production` он выключен, пока не задан `DEMO_AUTH_ENABLED=true` .
- **Права проверяются дважды:** в `store/useActions` для UX и на сервере при каждом сохранении (`authorize_state_replacement`). Клиент не может подменить чужие данные или служебные поля, например `telegram` .
- **Файлы:** белый список расширений, лимит 25 МБ, непрозрачные имена, скачивание только с токеном.
- **LLM локальная:** данные не уходят во внешние сервисы.
- **Руководитель** меняет, снимает и назначает ответственных своей команды; заявки без ответственного видит он и администратор.

Защита персональных данных (152-ФЗ и ФСТЭК № 117)

Требование	Как сделано
Шифрование ПДн при хранении (152-ФЗ, ст. 19)	AES-256-GCM (core/crypto.py): снимок CRM, история импортов, очередь Telegram — в БД только шифротекст; вложения шифруются потоком блоками по 1 МБ. Шифротекст привязан к полю (AAD): подмену или перестановку видно при чтении
Ключи	DATA_ENCRYPTION_KEYS в deploy/.env (0600), создаётся при деплое, хранится отдельно от данных. Ротация: новый ключ первым — при старте API всё перешифровывается (services/encryption.py). Production без ключа не запускается
Шифрование при передаче	HTTPS (Caddy, TLS) + HSTS
Защита веб-интерфейса	CSP (скрипты и запросы только к своему серверу и Keycloak), запрет встраивания во фреймы, nosniff , no-store для API (core/security_headers.py); шрифты и стили только локальные
Идентификация, аутентификация, пароли	Keycloak: пароль от 12 символов, история, смена раз в 90 дней, блокировка при подборе, сессия 30 мин
Управление доступом	Роли и области видимости, проверка на сервере при каждом сохранении
Регистрация событий	Журнал действий CRM, события входа и администрирования Keycloak — 180 дней
Минимизация и трансграничная передача (ст. 5, 12)	В Telegram — имя и инициал вместо ФИО, телефоны и почты в комментариях скрыты; явное согласие при подключении бота
Политика обработки ПДн (ст. 18.1)	Ссылка на экране входа и статья справки
Руководство администратора	В справке CRM: установка, ключ шифрования, резервные копии, журналы

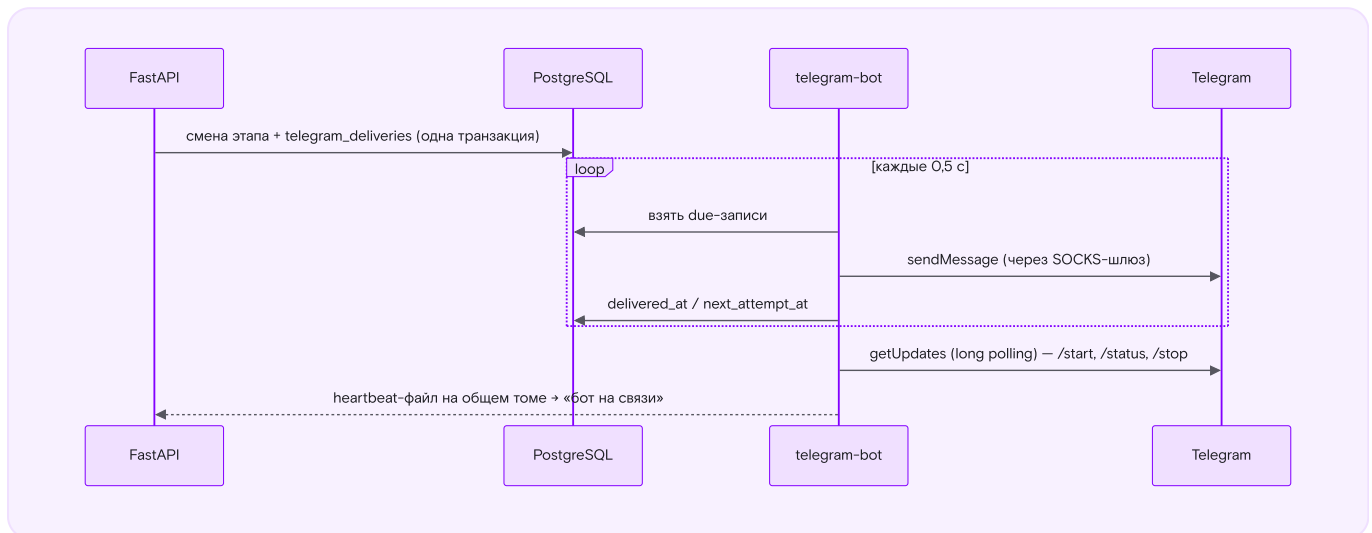
Для аттестации ГИС по приказу ФСТЭК № 117 криптозащиту каналов связи выполняют сертифицированными СКЗИ по ГОСТ (например, TLS-шлюз с ГОСТ-шифрованием перед Caddy); ключ шифрования данных может храниться в HSM. Код для этого менять не нужно.

ИИ-помощник изнутри



- Движок (`assistant/engine`) разбирает сущности по справочникам (падежи, сокращения), периоды и связки «кроме / мои / по ним».
- Модель не видит записи вузов и ничего не выполняет сама: она возвращает текст или одно действие, а выполняет его браузер.
- Если модель недоступна, всё, кроме свободного разговора, продолжает работать.

Telegram-бот изнутри



- Уведомление ставится в очередь с задержкой 1 с: если пользователь нажал «Отменить», сообщение не уйдёт.
- Получатели: ответственный, его руководитель и администраторы — если подключили чат.
- Привязка — одноразовая ссылка `t.me/<бот>?start=<код>` , живёт 15 минут. Chat ID хранится только на сервере.
- С одним токеном опрашивать Telegram может **только один** процесс. Локальный API запускайте с `TELEGRAM_POLLING_ENABLED=false` , иначе он будет выбивать продового бота (`409 Conflict`).

База данных

Таблицы, которые использует работающий API (миграции Alembic в [backend/alembic/versions](#)):

state_snapshots			
int	id	PK	всегда 1
json	state		снимок CRM
bigint	revision		optimistic locking
timestampz	updated_at		

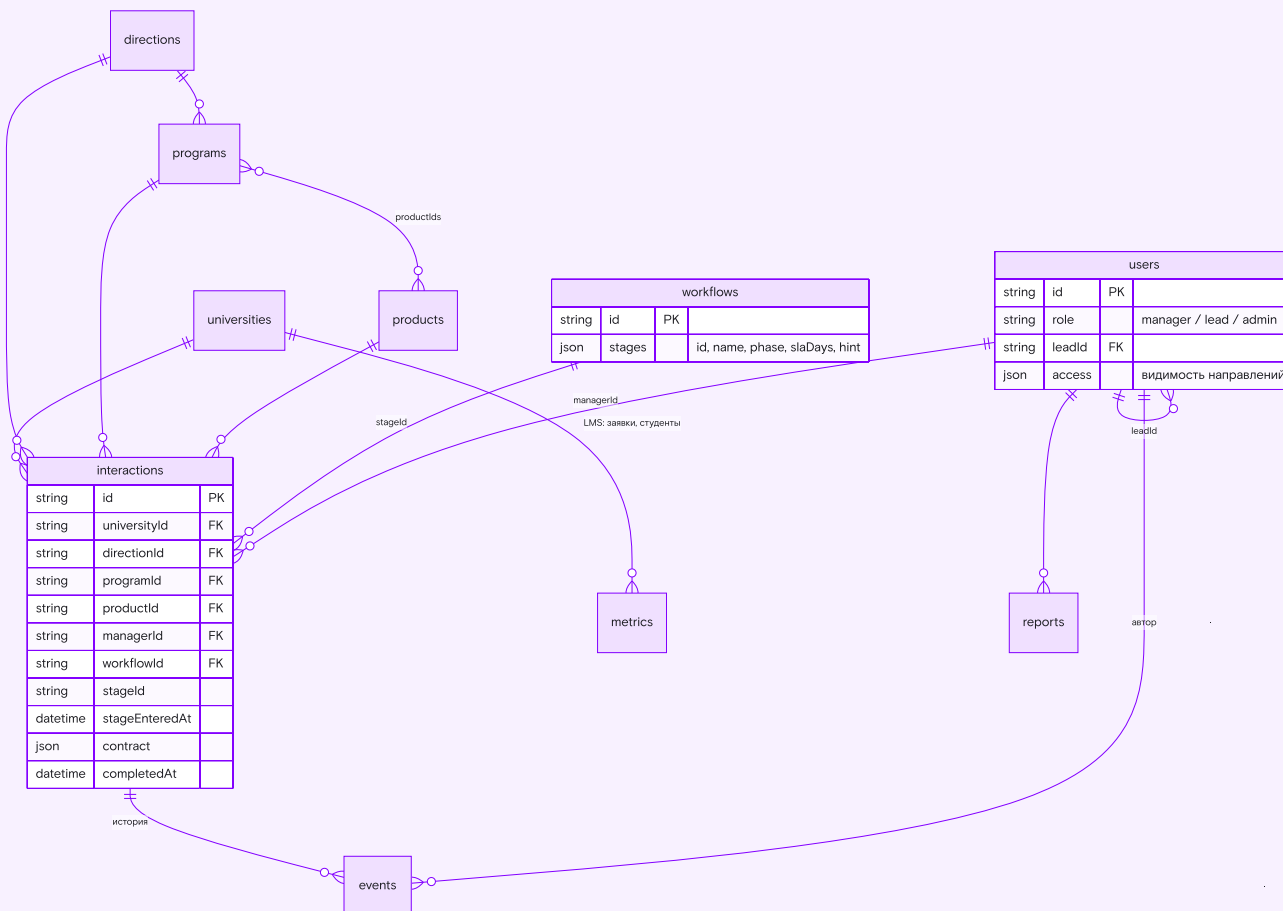
import_jobs			
string	id	PK	
string	file_name		
string	attachment_id	FK	
string	status		applied / rolled_back
json	stats		
json	issues		
json	snapshot_before		для отката
bigint	revision_after		
string	created_by		

telegram_deliveries			
string	id	PK	
string	event_id		
bigint	chat_id		
text	text		
timestampz	due_at		
timestampz	next_attempt_at		
int	attempts		
timestampz	delivered_at		

исходный файл

attachments			
string	id	PK	
string	original_name		
string	stored_name	UK	
string	content_type		
bigint	size		
string	uploaded_by		

Логическая модель внутри снимка:



Также в снимке лежат `inbox` (входящие из LMS и сайта), `integrations` и `audit`. Эскиз нормализованной PostgreSQL-схемы — `db.sql`.

API

Swagger: `/docs`, OpenAPI: `/openapi.json`. Все пути начинаются с `/api/v1`.

Группа	Эндпоинты
Состояние	<code>GET /state</code> , <code>POST /state/changes</code> (изменения клиента), <code>PUT /state</code> (целый снимок, совместимость), <code>POST /state/reset</code> , <code>GET /state/export</code> (результатирующий JSON)
Авторизация	<code>POST /auth/demo</code> , <code>GET /auth/me</code>
Профиль	<code>PUT /me/preferences</code> , <code>PUT /me/onboarding</code>
Сотрудники	<code>POST /accounts</code> , <code>PUT /accounts/{id}/role</code> , <code>.../status</code> , <code>POST .../password</code> — учётки в Keycloak
Файлы	<code>POST /attachments</code> , <code>GET /attachments/{id}</code>
Импорт	<code>POST /imports</code> , <code>GET /imports[/]{id}]</code> , <code>POST /imports/{id}/rollback</code>
Интеграции	<code>POST /integrations/{lms,site}/ingest</code> , <code>.../sync</code> , <code>GET /integrations/sources</code>
Помощник	<code>POST /assistant/chat</code> , <code>GET /assistant/status</code>
Telegram	<code>GET/DELETE /me/telegram</code> , <code>POST /me/telegram/link</code> , <code>POST /me/telegram/test</code> , <code>GET /telegram/status</code>
Только чтение	<code>GET /catalogs</code> , <code>/interactions</code> , <code>/workflows</code> , <code>/users</code> , <code>/audit</code> , <code>/integrations</code> , <code>/reports</code>
Служебное	<code>GET /health</code>

Ошибки приходят в едином формате `{"error": {"code", "message", "details"}}`. Коды описаны во встроенной справке.

Пример — сохранить комментарий к взаимодействию:

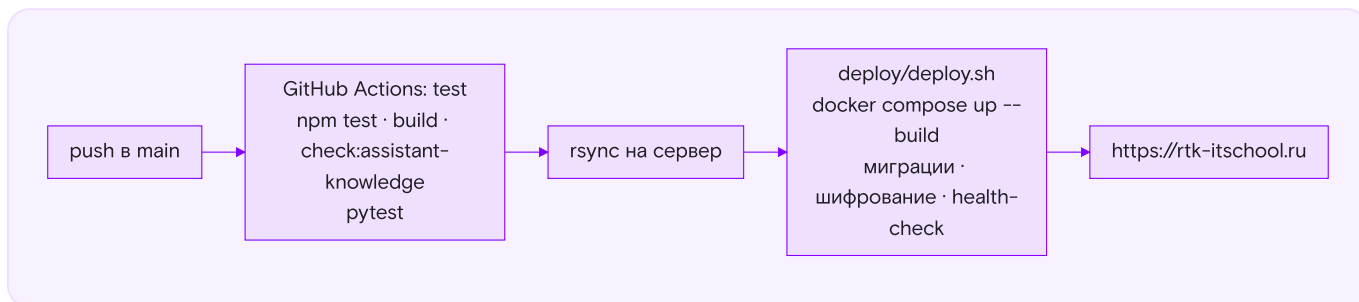
```
POST /api/v1/state/changes
Authorization: Bearer <токен>
Content-Type: application/json
```

```
{"changes": {"events": {"op": "list", "append": [{"id": "e-1", "interactionId": "i-1", "type": "comment", "userId": "usr-1", "at": "2026-09-29T10:00:00Z", "comment": "Созвонились", "files": []}]}}}
```

Формат изменений: `{"op": "list", "set", "prepend", "append", "remove"}` для списков, `{"op": "object", "fields"}` для вложенных объектов, `{"op": "value", "value"}` для остального. Демо-токен для Swagger: `POST /auth/demo` с `{"role": "admin"}`.

Эксплуатация

Развёртывание на сервере



Требования: Linux (Ubuntu 22.04+), Docker Engine и Docker Compose v2, 4 vCPU и 8 ГБ RAM (модель Ollama ~1–3 ГБ), открытые TCP 80 и 443. Наружу публикуется только Caddy; PostgreSQL, API, Keycloak и Ollama доступны лишь внутри сети Docker.

Первый запуск:

```
git clone <репозиторий> ~/rtk-it-school && cd ~/rtk-it-school
bash deploy/deploy.sh
```

Скрипт `deploy/deploy.sh` :

1. Один раз создаёт `deploy/.env` (права 0600, не в Git): пароли PostgreSQL и Keycloak, секрет JWT, ключ шифрования ПДн, секрет служебного клиента Keycloak. Адрес нового сервера задаётся в вызове `init.py http://<IP>` внутри скрипта.
2. Готовит realm Keycloak со случайными временными паролями сотрудников — они в `deploy/generated/initial-credentials.json`.
3. Собирает образы и запускает `deploy/compose.yml` : Caddy, API (+ собранный фронтенд), бот Telegram, SOCKS-шлюз для Telegram, PostgreSQL 16, Keycloak 26, Ollama.
4. При старте API применяет миграции Alembic и шифрует данные, записанные до включения шифрования.
5. Загружает модель Ollama, создаёт демо-учётку `testuser` и ждёт `GET /api/v1/health`.

Обновление — тот же `bash deploy/deploy.sh` (или push в `main`): данные, секреты и ключ не меняются.

Домен и HTTPS. А-запись домена → IP сервера, затем на сервере `python3 deploy/activate-domain.py` : скрипт переключит публичный адрес в `deploy/.env` и realm Keycloak, дождётся сертификата Let's Encrypt и откатит прокси, если сертификат не выдан.

CI/CD. GitHub Actions (`deploy.yml`) после push в `main` прогоняет тесты, копирует код по SSH и запускает `deploy.sh` . Нужен секрет репозитория `DEPLOY_SSH_KEY_V2` (приватный ключ, публичный — в `~/.ssh/authorized_keys` на сервере); ключ хоста закреплён в `deploy/known_hosts` .

Проверка:

```
curl -fsS https://<домен>/api/v1/health
docker compose --env-file deploy/.env -f deploy/compose.yml ps
docker compose --env-file deploy/.env -f deploy/compose.yml logs -f api telegram-bot
```

Резервные копии:

```
docker compose --env-file deploy/.env -f deploy/compose.yml exec -T postgres pg_dumpall -U rtk
> backup.sql
```

Плюс том `rtk-it-school_attachments` (файлы уже зашифрованы) и отдельно `deploy/.env` с ключом шифрования — без ключа данные не восстановить. Не выполняйте `docker compose down -v`: это удалит базы, вложения и модель.

Keycloak

Realm `it-school` (`realm-it-school.json`) импортируется при первом старте:

- Роли через группы: `kam` → менеджер, `manager_lead` → руководитель, `admin` → администратор.
- Клиенты: `rtk-it-school-web` — публичный, Authorization Code + PKCE (S256), без секрета; `rtk-it-school-api` — audience токенов; `rtk-it-school-admin` — служебный, CRM через него создаёт и блокирует учётки (только права на пользователей).
- Политики: пароль от 12 символов с разными типами знаков, история 5 паролей, смена раз в 90 дней; блокировка после 5 неудачных попыток; токен доступа 5 минут, сессия гаснет через 30 минут бездействия; журнал событий — 180 дней; самостоятельная регистрация выключена.
- Демо-учётка: `testuser` / `ItSchool-2026!` (администратор). Остальные сотрудники — `a.voronova`, `m.orlov`, `e.kim`, `d.sokolov`, `o.lebedeva` (менеджеры), `a.kozlov`, `n.belova` (руководители); локально пароль у всех `ItSchool-2026!`.
- Сотрудники заводятся в CRM («Пользователи и доступ» → «Добавить сотрудника»): CRM создаёт учётку в Keycloak и один раз показывает временный пароль; при первом входе Keycloak попросит задать свой.

Локально без Docker (Java 21): `backend/deploy/keycloak/run-local.sh` — поднимет Keycloak на `http://localhost:8080` (консоль `admin` / `admin`).

Telegram-бот

1. В `@BotFather` отправьте `/newbot` — он пришлёт токен.
2. Укажите `TELEGRAM_BOT_TOKEN=...` в `deploy/.env` (локально — в `backend/.env.local`, файл не в Git) и перезапустите. Для кнопки «Открыть карточку» задайте `PUBLIC_APP_URL` (не localhost).
3. Сотрудник: «Настройки профиля» → «Подключить Telegram» → «Старт» в Telegram. Состояние бота — на странице «Интеграции».

С одним токеном опрашивать Telegram может только один процесс: локальный API запускайте с `TELEGRAM_POLLING_ENABLED=false`. Если Telegram из сети сервера недоступен, задайте

TELEGRAM_PROXY_URL (`http://` или `socks5://`); в Docker по умолчанию используется встроенный шлюз `telegram-egress`.

Переменные окружения

Полный список — `backend/.env.example` и `frontend/.env.example`. Главные:

Переменная	Назначение
APP_ENV	<code>development</code> / <code>demo</code> / <code>production</code> ; в production API требует токен и ключ шифрования
DATABASE_URL	Строка подключения, например <code>postgresql+psycopg://user:pass@host:5432/db</code> ; по умолчанию SQLite
DATA_ENCRYPTION_KEYS	Ключи AES-256 для ПДн: <code>id:ключ</code> , при ротации новый первым
JWT_SECRET, DEMO_AUTH_ENABLED	Подпись демо-токенов и включение демо-входа
KEYCLOAK_ISSUER_URL, KEYCLOAK_AUDIENCE, KEYCLOAK_CLIENT_ID	Проверка токенов Keycloak
KEYCLOAK_ADMIN_CLIENT_ID, KEYCLOAK_ADMIN_CLIENT_SECRET	Служебный клиент для учёток из CRM
LMS_API_URL, WEBSITE_API_URL (+ _TOKEN), INTEGRATION_DEMO_ENABLED	Источники LMS и сайта
OLLAMA_BASE_URL, OLLAMA_MODEL, ASSISTANT_ENABLED	Локальная модель помощника
TELEGRAM_BOT_TOKEN, TELEGRAM_POLLING_ENABLED, TELEGRAM_PROXY_URL, PUBLIC_APP_URL	Telegram-бот
VITE_KEYCLOAK_URL, VITE_KEYCLOAK_REALM, VITE_KEYCLOAK_CLIENT_ID	Сборка фронтенда: кнопка «Войти через Keycloak»

Локальный запуск

Нужны Python 3.12+ и Node.js 22+.

```
cd backend && python3 -m venv .venv-local && .venv-local/bin/pip install -r requirements.txt &&
cd ..
backend/run-local.sh           # сборка фронтенда + API на http://localhost:8000 (Swagger —
/docs)
```

Разработка с горячей перезагрузкой:

```
cd backend && TELEGRAM_POLLING_ENABLED=false .venv-local/bin/uvicorn app.main:app --reload --port 8000
cd frontend && npm install && npm run dev # http://localhost:5173, /api проксируется на :8000
```

Весь стенд в Docker (API, PostgreSQL, Keycloak): `cd backend && docker compose up --build` — приложение на <http://localhost:8000>. Помощнику нужен Ollama: `ollama pull qwen3:4b-instruct`.

Тесты

Команда	Что проверяет
<code>cd frontend && npm test</code>	98 юнит-тестов: доменная логика, синхронизация, чтение XLS (<code>node:test</code>)
<code>cd backend && python -m pytest -q</code>	65 тестов: API, права, шифрование ПДн и его миграция, заголовки, импорт, Telegram

Перед релизом дополнительно проводились сквозные проверки на отдельном стенде: 109 проверок API, 83 UI-сценария, обход 1681 элемента интерфейса тремя ролями, 15 одновременных отчётов и нагрузка до 150 пользователей (0 ошибок, р95 записи 117 мс).

Библиотеки и компоненты

Где	Библиотека / компонент	Зачем
Backend	FastAPI, Uvicorn	REST API, Swagger UI (/docs)
	SQLAlchemy 2, Alembic, psycopyg 3	ORM, миграции, драйвер PostgreSQL
	Pydantic Settings	Конфигурация из окружения
	PyJWT, cryptography	Проверка токенов Keycloak (RS256), AES-256-GCM для ПДн
	python-multipart, httpx[socks]	Загрузка файлов; запросы к LMS, сайту, Ollama, Telegram
	pytest	Тесты
Frontend	React 19, Vite 8, @vitejs/plugin-react	Интерфейс и сборка
	@atomaro/ui-kit, @atomaro/icons, @base-ui/react, styled-components	Дизайн-система Ростелекома, доступные диалоги и меню
	GSAP	Анимации
	@noble/hashes	SHA-256 для РКСЕ при входе через Keycloak

Где	Библиотека / компонент	Зачем
Собственные модули	PDF, XLSX, XLS, ZIP, PNG-графики, чтение XLSX/XLS/CSV	Без сторонних библиотек: <code>frontend/src/lib</code>
Инфраструктура	PostgreSQL 16, Keycloak 26.7, Ollama (Qwen), Caddy 2.10, Docker Compose	БД, SSO, локальная LLM, HTTPS

Структура репозитория

frontend/	SPA: <code>src/{app,api,auth,domain,store,ui,layout,features,lib,styles}</code> , <code>tests/</code> ,
scripts/	
design/	дизайн-правила и шрифт Rostelecom Basis
backend/	FastAPI: <code>app/{api,core,services,models,schemas,domain}</code> , <code>alembic/</code> , <code>tests/</code>
deploy/keycloak	realm Keycloak и локальный запуск
deploy/	продакшен: <code>compose</code> , <code>Caddy</code> , домен, Telegram-шлюз
logic/	правила процесса (этапы и переходы)
docs/	медиа README, диаграмма репозитория, модель Archi (<code>architecture/</code>)